

FlashShield: Detecting Flash Loan Attacks in DeFi Using Hypergraph Neural Network

Xinpeng Huang[✉], Wangjie Qiu[✉], Wanqing Jie, Qing Xia, Qinnan Zhang[✉],
Yuqiang Sun[✉], *Graduate Student Member, IEEE*, Maoyi Xie[✉], Yang Liu[✉], *Senior Member, IEEE*,
and Zhiming Zheng

Abstract—The rapid growth of decentralized finance (DeFi) has spurred innovation but also exposed blockchain systems to severe security threats. As of November 2025, cumulative losses from blockchain security incidents have exceeded \$36.89 billion. Flash loan attacks account for 135 reported cases and rank fourth among all attack methods. Existing detection approaches either analyze contract source code, which is unavailable for many deployed contracts, or use transaction pattern matching tailored to specific scenarios, and therefore generalize poorly to diverse flash loan attacks. In this paper, we present *FlashShield*, a general flash loan attack detection framework based on Hypergraph Neural Networks (HGNNs). We construct comprehensive datasets containing attack and benign transactions across multiple chains, and systematically analyze flash loan attack mechanisms along four DeFi protocol layers: code implementation, business logic, economic mechanisms, and cross protocol interactions. *FlashShield* represents each transaction as a hypergraph of transfer actions and semantic relations, and employs a hybrid architecture that integrates spectral, spatial, and original features together with both node level and graph level representations. Experiments show that *FlashShield* improves recall by 29% over leading methods and identifies 43 previously unknown malicious or suspicious activities (18 confirmed flash loan-related exploits and 25 suspected address poisoning incidents), demonstrating its effectiveness and scalability for automated DeFi security monitoring.

Index Terms—Blockchain, DeFi, flash loan, attack detection, hypergraph.

Received 12 December 2025; revised 6 April 2026; accepted 25 May 2026. Date of publication 29 May 2026; date of current version 1 September 2026. This work was supported by the National Cyber Security-National Science and Technology Major Project under Grant 2025ZD1501002, in part by the National Natural Science Foundation of China under Grant 62572029, in part by the Beijing Natural Science Foundation under Grant Z230001, in part by the Beihang Ganwei Action Plan under Grant JKF-2025083360604, in part by the Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing, and in part by the China Scholarship Council (CSC). (Corresponding authors: Wangjie Qiu; Qinnan Zhang.)

Xinpeng Huang, Wangjie Qiu, Wanqing Jie, and Zhiming Zheng are with the Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing, School of Artificial Intelligence, Beihang University, Beijing 100191, China, and also with Zhongguancun Laboratory, Beijing 100190, China (e-mail: huangxp@buaa.edu.cn; wangjieqiu@buaa.edu.cn; wanqingjie@buaa.edu.cn; zzheng@pku.edu.cn).

Qing Xia is with the Institute of Software, Chinese Academy of Sciences, Beijing 100864, China (e-mail: xiaqing2018@iscas.ac.cn).

Qinnan Zhang is with the Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing, School of Artificial Intelligence, Beihang University, Beijing 100191, China (e-mail: zhangqn@buaa.edu.cn).

Yuqiang Sun, Maoyi Xie, and Yang Liu are with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: suny0056@e.ntu.edu.sg; maoyi001@e.ntu.edu.sg; yangliu@ntu.edu.sg).

Digital Object Identifier 10.1109/TDSC.2026.3698115

I. INTRODUCTION

AS A form of distributed ledger technology, blockchain has been widely adopted in domains such as finance and energy due to its properties of immutability, transparency, and decentralization [1]. Among these applications, decentralized finance (DeFi) has evolved into a vibrant ecosystem encompassing various tokens and protocols, including flash loans, decentralized exchanges (DEXs), and lending platforms. However, the rapid expansion of DeFi has also brought frequent security incidents. As of November 2025, cumulative losses from blockchain security incidents have exceeded \$36.89 billion [2]. Flash loan attacks account for 135 reported incidents, making them the fourth most frequent attack method [2]. Moreover, flash loan attacks are often combined with other exploits, such as smart contract vulnerabilities, which remain the most common category [3].

Flash loan protocols inherently expose DeFi systems to high risks. As a form of unsecured loan, flash loans allow users to borrow large amounts of crypto assets without collateral, provided that the borrowed funds are repaid within a single transaction [4]. This feature enables attackers to manipulate market prices or exploit contract flaws within one atomic transaction for profit [5]. Understanding such attack mechanisms and developing effective defenses are thus crucial for ensuring DeFi ecosystem stability.

Existing studies on flash loan attack detection can be grouped into two categories: code-based and transaction-based. Code-based methods analyze smart contract source code to identify vulnerabilities or malicious behaviors, but their applicability is limited since some contracts lack accessible code [6]. Transaction-based methods, in contrast, analyze fund flows and behavioral patterns from public blockchain data to infer attack intent. However, most of these methods focus on specific attack types, such as price manipulation [5], [6], [7], [8], [9], while neglecting other forms of flash loan abuse [3]. Although the latest work [3] proposed a taxonomy of root causes, its classification mainly emphasizes smart contract vulnerabilities rather than the attack mechanisms across DeFi protocols [10]. Since flash loan attacks may involve multiple layers of DeFi protocols, including code implementation, business logic, cross-protocol interaction, and economic mechanisms, an integrated analysis over these dimensions is required to detect them effectively. Nevertheless, a general transaction-based detection framework capable of identifying diverse flash loan attacks remains absent, and designing such a system poses several key challenges.

Challenge 1. Diversity and evolution of flash loan attacks: Flash loan attacks exhibit diverse and evolving patterns. For example, price manipulation and vulnerability exploitation attacks differ significantly in their transactional behaviors: the former manipulates market prices through large trades to gain profit, while the latter exploits vulnerabilities in smart contracts. Even within a single attack type, behavioral patterns evolve over time. For instance, rule-based detection methods such as DeFiScope [6] expanded the number of summarized attack patterns from 3 (as in Leishen [5]) to 8. These rule-based approaches rely on predefined behavioral templates, making them suitable only for specific attack types and prone to obsolescence.

Challenge 2. Data imbalance due to limited flash loan attack samples: Compared to benign transactions, flash loan attack samples are extremely scarce, leading to severe class imbalance. This scarcity limits approaches based on deep learning, explaining why most existing methods remain choose pattern matching.

Challenge 3. Atomicity and behavioral complexity: Flash loans execute all operations within a single atomic transaction, which may encompass hundreds of interdependent actions spanning borrowing, manipulation, and repayment. This atomicity, combined with the behavioral diversity of exploit patterns, makes it difficult to distinguish malicious intent from legitimate complex usage. Hence, it is essential to aggregate asset transfer actions within a transaction and extract abstract semantic features to identify malicious intent.

To tackle these challenges, we construct comprehensive datasets covering both attack and benign flash loan transactions, model attack mechanisms crossing layers of DeFi protocols, and develop a general detection framework, *FlashShield*. Our work proceeds in three parts as follows.

Dataset construction: We aggregate data from multiple sources, including the public *REKT* incident database [11], GitHub repositories [12], and academic papers [5], [6], [7] (Section IV-A, addressing **Challenge 2**). Each attack case is manually verified to ensure consistency with disclosed security incidents. Additionally, 12,102 Ethereum flash loan transactions (July 29, 2023-August 06, 2025) are collected for large-scale evaluation. Eventually, we constructed a mixed dataset comprising both attack and benign samples, a dataset consisting exclusively of benign transactions, and a large-scale real-world dataset.

Attack mechanism analysis: We establish a four-layer attack mechanism model: code, economic, business logic, and cross-protocol mechanisms (Section III-B, addressing **Challenge 1**), breaking the conventional *vulnerability-centric* classification paradigm. Case studies reveal how attackers exploit flash loans as zero collateral leverage tools to amplify local weaknesses and gain profit. We further identify a new attack type, *Cross-Protocol Mechanism Dislocation (CPMD)*, exposing systemic risks from inconsistencies in valuation, state synchronization, or liquidation logic across DeFi protocols.

Proposed detection framework: *FlashShield* employs the hypergraph neural networks (HGNNs) for transaction behavior analysis (Section IV-B, addressing **Challenge 3**). Each fund transfer action is modeled as a node, capturing complex

fund flows via *action-centric hypergraph modeling*. *Semantic-structural hyperedge construction* integrates semantic relations (shared addresses, tokens, temporal links) and structural similarity to detect coordinated multi-protocol behaviors. Finally, *spectral-spatial-original hybrid feature integration* fuses global and local representations, and *node-graph feature fusion* combines node-level and graph-level embeddings for robust detection of structural and semantic anomalies.

To evaluate the proposed framework, we first compare *FlashShield* with state-of-the-art methods on the mixed dataset, jointly assessing attack recall and false positives. *FlashShield* already exhibits a very low false-positive rate in this setting, so we further examine its false-positive behavior on a benign-only dataset and analyze the causes of the remaining false alarms. We then perform ablation studies on *FlashShield*'s main modules and conduct a large-scale real-world evaluation on the third dataset. Experimental results show that *FlashShield* achieves an F1 score of 98.36% and a recall of 96.77%, improving recall by 29% over the best existing method. Moreover, the large-scale evaluation uncovers 43 previously unknown flash loan attacks.

The main contributions of this work are as follows:

- 1) *Four-layer flash loan attack taxonomy:* Beyond prior work focusing on price manipulation, we systematically analyze flash loan attacks across four DeFi protocol layers (code, business logic, economic, cross-protocol) and reveal how this taxonomy directly informs our detection framework design, including semantic-structural hyperedge construction (Section IV-B), global feature engineering (Section IV-A), and multi-scale feature fusion (Section IV-B).
- 2) *General flash loan attack detection framework FlashShield:* We design *FlashShield* based on hypergraph neural networks (HGNNs). *FlashShield* performs *action-centric hypergraph modeling* by treating individual transfer actions as nodes and employs *semantic-structural hyperedge construction* to capture complex semantic relations and coordinated multi-protocol behaviors. A hybrid architecture combining *spectral-spatial-original feature integration* and *node-graph feature fusion* enables hierarchical representation learning for robust detection of structural and semantic anomalies.
- 3) *Datasets and experimental evaluation:* We construct three flash loan transaction datasets. Experiments on these datasets demonstrate that *FlashShield* achieves an F1 score of 98.36% and a recall of 96.77%, improving recall by 29% over the best existing method. In addition, our large-scale evaluation uncovers 43 previously unknown malicious or suspicious activities, comprising 18 confirmed flash loan-related exploits and 25 suspected address poisoning incidents. Our code is available at: <https://github.com/JustNSim/FlashShield-public>.

The remainder of this paper is organized as follows. Section II reviews the relevant background and related work. Section III addresses the problem definition and mechanism analysis. Section IV details the proposed approach *FlashShield*. Section V presents the experimental evaluation results. Section VI concludes the paper.

II. BACKGROUND AND RELATED WORK

A. Blockchain and DeFi

Blockchain: Public blockchains such as Ethereum [13] use distributed ledger technology to immutably record transactions. They distinguish between two account types: *Externally Owned Accounts* (EOAs) controlled by users via private keys, and *Contract Accounts* governed by autonomous smart contract code. Transactions originating from EOAs can invoke Contract Accounts, which may trigger cascading contract interactions before atomic completion. This account model enables DeFi protocols to operate without intermediaries.

DeFi: Decentralized Finance (DeFi) is an open, permissionless financial ecosystem built on public blockchains [14], [15]. Unlike traditional centralized finance (CeFi), which relies on banks and brokers to custody assets and enforce rules, DeFi encodes financial logic directly into smart contracts (self-executing programs that enforce transaction terms without trusted third parties). This architecture ensures transparent on-chain audibility and enables global, permissionless access.

Several DeFi primitives are foundational to understanding flash loan attacks. *Automated Market Makers (AMMs)* [16] (e.g., Uniswap, SushiSwap) replace traditional order books with smart contract-managed liquidity pools priced by algorithmic formulas (e.g., $x \times y = k$), making price movements predictable and exploitable. *Lending protocols* [4] (e.g., Aave, Compound) let users borrow assets against over-collateralized deposits at algorithmically set rates. *Flash loans* [17] extend this by enabling uncollateralized borrowing within a single atomic transaction: if repayment does not occur before transaction close, the entire transaction reverts, providing attackers with large-scale capital at zero upfront cost.

B. DeFi Transaction Monitoring and Attack Detection

The rapid expansion of DeFi has spurred significant interest in transaction monitoring and security analysis, as the immutable and open nature of blockchain also exposes vulnerabilities to exploitation. Current research efforts primarily concentrate on two complementary directions:

(1) *Account de-anonymization*: Although all blockchain data are publicly accessible, user identities remain pseudonymous, making it challenging to link addresses to real-world entities. To address this, graph-based analytical techniques have been developed to uncover latent relationships among addresses [18], [19]. These methods leverage transaction flows, contract invocation patterns, and behavioral signatures to reconstruct organizational or operational entities operating on the blockchain.

(2) *DeFi attack detection*: Machine learning, particularly Graph Neural Networks (GNNs) [20], has been extensively employed to detect anomalous or malicious activities in DeFi transactions. Existing approaches target a variety of threats, including attacks on cross-chain bridges [21], [22], address poisoning schemes that manipulate user interactions [23], exploitations of logic flaws in smart contracts [24], and phishing campaigns that deceive users into transferring funds [25], [26]. In addition to attack-specific detectors, recent work has

introduced general-purpose frameworks capable of large-scale transaction analysis and detection of diverse attack types on the blockchain. Examples include automated incident investigation systems [27] and pretrained models designed for broad-spectrum anomaly detection in DeFi environments [28].

C. Flash Loan Attack Detection

Given the security risks posed by flash loans, Qin et al. [17] quantitatively analyzed profit mechanisms in two attack case studies and proposed optimized attack strategies. Subsequent research has focused on detection, which can be broadly categorized into code-based and transaction-based approaches.

1) *Code-based detection*: DeFiTainter [8] employs byte-code taint analysis to detect price oracle manipulation vulnerabilities. FLASHSYN [9] synthesizes profitable flash loan attack vectors via mathematical approximation, while ProMutator [29] simulates oracle manipulation locally to assess protocol susceptibility.

2) *Transaction-based detection*: Most methods rely on pattern matching. DeFiRanger [30] constructs a *cash flow tree* from transaction data to recover DeFi semantics and detect oracle manipulation. Leishen [5] empirically studies flash loan price manipulation, summarizing three attack patterns and implementing corresponding rules. DeFort [7] tracks token price fluctuations and profit strategies to flag abnormal trading behaviors.

The latest framework, DeFiScope [6], combines code and transaction analysis. It uses large language models (LLMs) to abstract price computation logic from smart contracts and applies eight summarized manipulation patterns for more effective detection.

Table I summarizes the key differences between existing approaches and *FlashShield*. As shown, all prior methods are confined to price manipulation (PM) and most require access to contract source code or ABI, which is unavailable for many deployed protocols. In contrast, *FlashShield* operates solely on transaction data, covers all four attack types formally defined in Section III-B (Contract Vulnerability Exploitation (CVE), Business Logic Flaw (BLF), Price Manipulation (PM), and Cross-Protocol Mechanism Dislocation (CPMD)), and employs hypergraph neural networks to capture complex multi-party fund-flow semantics that flat or tree-structured representations cannot express.

III. PROBLEM DEFINITION AND MECHANISM ANALYSIS

In this section, we introduce the problem definition of flash loan attack detection, data collection, and Flash Loan Attack Mechanism Analysis and Categorization.

A. Problem Definition

Flash loan attacks refer to a type of attack that exploits vulnerabilities in the code or economic mechanisms of DeFi protocols to profit from a single blockchain transaction through the use of flash loan protocols.

The attack model for flash loan attacks is illustrated in Fig. 1. Unlike previous models that have revealed flash loan attacks [3],

TABLE I
COMPARISON OF FLASH LOAN ATTACK DETECTION APPROACHES (✓ = SUPPORTED; ✗ = NOT SUPPORTED)

Method	Category	Contract Required	Technique	Attack Coverage				Graph Model
				CVE	BLF	PM	CPMD	
DeFiTainter [8]	Code-based	✓	Taint analysis	✗	✗	✓	✗	None
FlashSyn [9]	Code-based	✓	Numerical approximation	✗	✗	✓	✗	None
ProMutator [29]	Code-based	✓	Transaction mutation	✗	✗	✓	✗	None
DeFiRanger [30]	Txn-based	✗	Cash flow tree + semantics	✗	✗	✓	✗	Tree
Leishen [5]	Txn-based	✗	Rule-based matching	✗	✗	✓	✗	None
DeFort [7]	Txn-based	✗	Behavior modeling	✗	✗	✓	✗	None
DeFiScope [6]	Hybrid	✓	LLM + rule mining	✗	✗	✓	✗	Graph
FlashShield (Ours)	Txn-based	✗	HGNN	✓	✓	✓	✓	Hypergraph

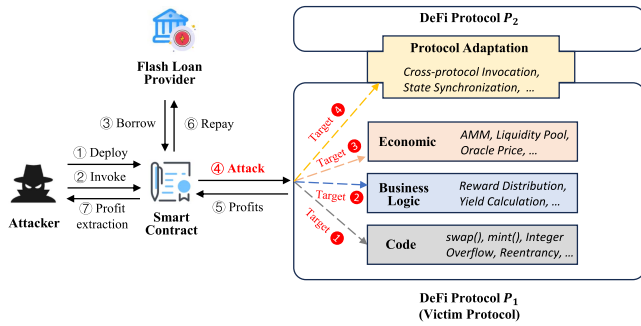


Fig. 1. The threat model of flash loan attacks across multi-layer DeFi protocols.

[5], we emphasize the actions and interactions across multiple layers, including code, logic, economic mechanisms, and cross-protocol aspects. This is partly due to the fact that flash loan transactions indeed involve complex events across these dimensions, which aligns with the multi-layered nature of current DeFi protocols [10]. On the other hand, this categorization also serves as a key basis for our classification of flash loan attacks (Section III-B).

In this attack model, the attack is divided into seven critical steps:

- 1) The attacker deploys an attack smart contract, which contains the code that implements the attack logic.
- 2) The attacker initiates a transaction that calls the smart contract to trigger the flash loan transaction.
- 3) Within the flash loan transaction, the smart contract borrows funds from a flash loan provider.
- 4) Using the borrowed funds, the smart contract exploits vulnerabilities or flaws in the victim DeFi protocol's code, business logic, economic mechanisms, or interaction design.
- 5) The attacking smart contract successfully realizes an illicit profit.
- 6) The smart contract repays the flash loan to the previous flash loan provider.
- 7) The attacker transfers the profits to external addresses under their control, completing the exploitation cycle.

B. Flash Loan Attack Mechanism Analysis and Categorization

To address *Challenge 1* (the diversity and evolution of flash loan attacks), we systematically analyze historical flash loan incidents and security reports to develop a comprehensive understanding of attack mechanisms. This analysis establishes the foundation for subsequent feature representation and model design. From this study, we propose a four-layer attack mechanism model encompassing code implementation, business logic, economic mechanisms, and cross-protocol mechanisms. Unlike prior classification methods based solely on vulnerability types [3], we adopt a multi-layer operational view of DeFi protocols. This approach systematically characterizes how attackers exploit flash loans to amplify localized weaknesses for arbitrage. Table II summarizes the classification criteria and definitions, followed by detailed discussions of each attack type, its mechanism, and representative cases.

1) *Contract Vulnerability Exploitation (CVE)*: This type refers to attackers exploiting code-level vulnerabilities in smart contracts, leveraging flash loans to amplify attack effects. Such attacks trigger abnormal contract calls, alter state variables, or bypass restrictions to extract assets. Common patterns include reentrancy attacks [31], missing access control [32], and unsynchronized accounting states [33].

Example (Reentrancy Exploitation): Fig. 2 illustrates a typical contract vulnerability exploitation path. The core mechanism is to disrupt the contract's state update order through reentrancy, thereby gaining abnormal rewards in the reward distribution logic. The attacker first borrows 70,000 ETH from dYdX (Step 1), swaps them for USDT and DAI, and deposits 7.5 million USDT into the OUSD Vault, invoking `mint()` and triggering `rebase()`, which grants the attacker more than 50% ownership (Steps 2–3). In Step 4, the attacker calls `mintMultiple()`, and during the `transferFrom()` stage, constructs a reentrant call to the Vault's `mint()` function, re-triggering `rebase()`. Because the contract state has not yet synchronized, the system mistakenly allocates 20.5 million DAI as unassigned yield, giving the attacker over \$10.25 million in false rewards (Step 5). After the reentrancy process ends (Steps 6–7), the attacker redeems and swaps OUSD, obtaining control of over \$38.25 million in assets, while the vault holds

TABLE II
DEFINITION OF FOUR TYPES OF FLASH LOAN ATTACKS

Analysis Dimension	Flash Loan Attack	Definition
Code Implementation	Contract Vulnerability Exploitation (CVE)	Attackers use flash loans to rapidly invoke contract functions with code vulnerabilities (e.g., reentrancy, privilege bypass), triggering unintended state changes.
Business Logic	Business Logic Flaw (BLF)	Attackers exploit design flaws in business logic to amplify mechanism defects such as reward distribution or mint-redeem processes, extracting assets via legitimate interaction paths.
Economic Mechanism	Price Manipulation (PM)	Attackers manipulate a protocol's price mechanism within a short period using flash loans, influencing valuation, liquidation, or reward logic to obtain abnormal profits.
Cross-Protocol Mechanism	Cross-Protocol Mechanism Dislocation (CPMD)	Attackers exploit valuation differences, price update delays, or precision mismatches among protocols, orchestrating cross-protocol operations to create dislocations and arbitrage opportunities.

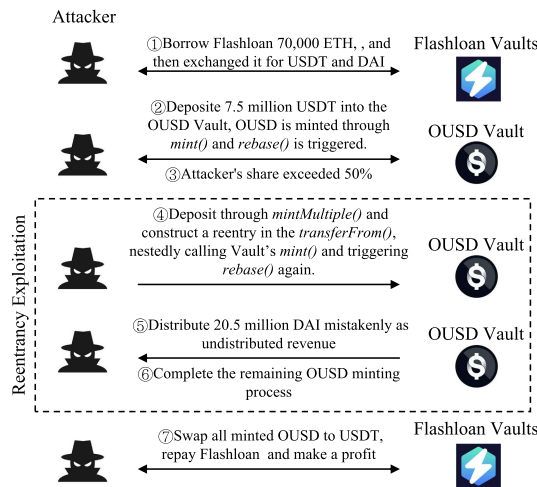


Fig. 2. An example of Contract Vulnerability Exploitation (Reentrancy Attack).

only \$35.01 million. The arbitrage is fully completed within a single transaction.

2) *Business Logic Flaw (BLF)*: This type refers to attackers exploiting design flaws in business logic using flash loans to amplify operations, triggering abnormal states for profit. Unlike code vulnerabilities, these flaws arise from poor mechanism design in reward distribution, mint-redeem, asset accounting, or airdrop logic. Common modes include reward mechanism flaws [34], minting logic flaws [35], fake-token bypass [36], and precision miscalculation [37].

Example (Reward Mechanism Flaw): Fig. 3 illustrates an arbitrage path based on a reward distribution flaw. Its core mechanism is to manipulate staking ratios and performance-token minting rules to amplify incentives. The attacker borrows 1,615,059 CAKE from BiSwap and PancakeSwap via flash loans (Step 1), deposits 509,143 CAKE into ApeRocket AutoCake Pool (Step 2), and injects 1,105,916 CAKE as rewards (Step 3). With a dominant stake, nearly all rewards accrue to the attacker (Step 4). Upon calling `withdrawAll()`, the system deducts fees and compensates with \$SPACE tokens via `mintFor()` (Step 5). Since the mint multiplier is hard-coded as 64×10^{18} , vastly exceeding the real price, \$SPACE is severely overissued (Step 6). Finally, the attacker swaps \$SPACE for WBNB,

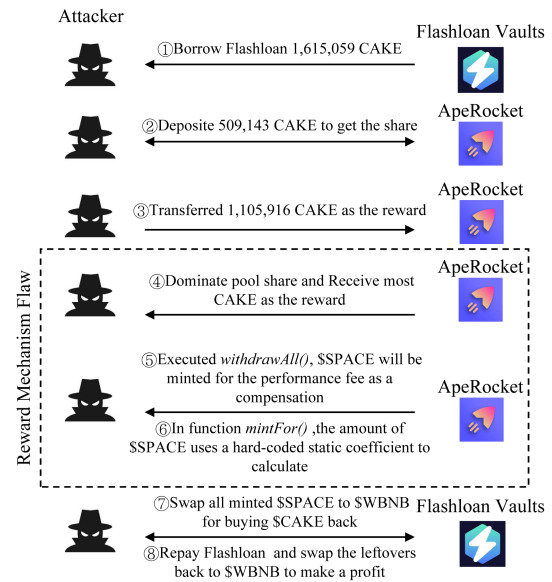


Fig. 3. An example of business logic flaw (reward mechanism flaw).

repurchases CAKE, repays the flash loans, and realizes profit (Steps 7-8).

3) *Price Manipulation (PM)*: This type refers to attackers manipulating the price mechanism of a single DeFi protocol at the economic mechanism level via flash loans, misleading DeFi protocols in valuation, liquidation, or reward distribution logic to achieve improper arbitrage. Common patterns include pump arbitrage [38], oracle price manipulation [39], dump-liquidation arbitrage [40], and liquidity pool price distortion arbitrage [41].

Example (Pump Arbitrage): Fig. 4 shows a typical price manipulation attack path *pump arbitrage*, whose core mechanism is to use flash loans to push up AMM quotes, thereby constructing an overvalued arbitrage opportunity. The attacker first borrows 10,000 ETH from dYdX via a flash loan (Step 1). Then, 5,500 ETH are deposited into Compound as collateral to borrow 112 WBTC (Step 2). In Step 3, the attacker opens a leveraged position on bZx using 1,300 ETH to build a five-times margin position and swaps 5,637 ETH for 51.35 WBTC through KyberSwap. As the transaction routes through Uniswap, the WBTC/ETH rate jumps from approximately 38.5 ETH/WBTC to 109.8 ETH/WBTC. Before the price reverts, the attacker

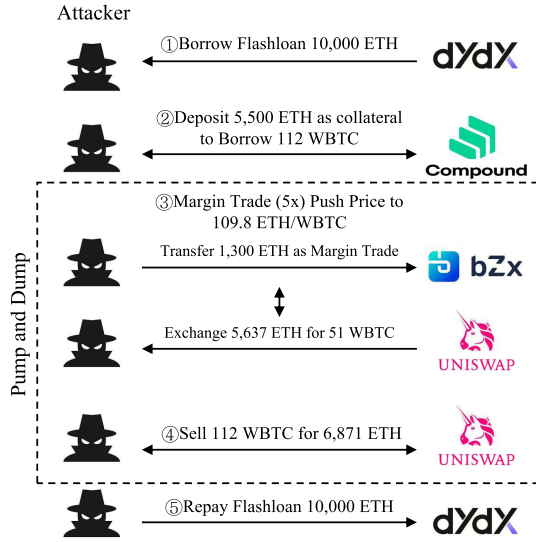


Fig. 4. An example of price manipulation (pump and dump).

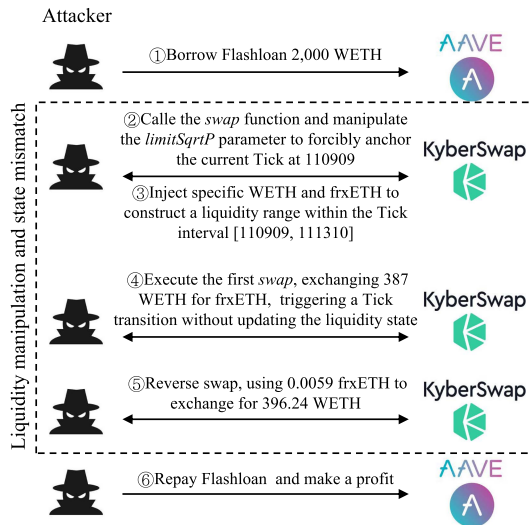


Fig. 5. An example of cross-protocol mechanism dislocation (cross-chain liquidity manipulation).

quickly sells the previously borrowed 112 WBTC at the inflated price, receiving 6,871.41 ETH (average 61.4 ETH/WBTC) (Step 4). Finally, the attacker repays the 10,000 ETH flash loan to dYdX (Step 5), completing the transaction loop and realizing a profit.

4) *Cross-Protocol Mechanism Dislocation (CPMD)*: This type refers to attackers executing high-frequency interactions across multiple DeFi protocols, leveraging inconsistencies in valuation, liquidation rules, price updates, or share calculation mechanisms to create arbitrage opportunities. Due to structural differences among protocols, attackers can exploit state mismatches and timing delays to shuttle assets and amplify value. Common patterns include cross-protocol valuation arbitrage [42], liquidity migration arbitrage [43], governance manipulation arbitrage [44], and cross-chain liquidity control [45].

Example (Cross-Chain Liquidity Manipulation): Fig. 5 illustrates a typical *cross-protocol dislocation* attack. The core

mechanism is to inject and manipulate local liquidity structures within specific tick ranges and exploit inconsistent state updates for arbitrage. The attacker borrows 2,000 WETH via an Aave V3 flash loan (Step 1), then performs a small swap to anchor the pool price at Tick 110909, injecting WETH and frxETH to construct a liquidity range of [110909, 111310] (Steps 2–3). In Step 4, the attacker swaps 387 WETH, moving the price to Tick 111310. However, due to improper liquidity deduction, the variable `currentTick` is erroneously updated, skipping intermediate state transitions. Then, in Step 5, a reverse swap uses only 0.0059 frxETH to obtain 396.24 WETH, as prior liquidity remained incorrectly active. The attacker completes the arbitrage loop within a single transaction, realizing significant profit.

Design Implications: The four-layer analysis above guides our framework design in Section IV. **Code-layer** contract dependencies inform *address hyperedges*; **Business Logic-layer** atomicity motivates *temporal hyperedges*; **Economic-layer** price manipulations drive *flow-chain hyperedges* and amount-related global features (mean, max, Gini); **Cross-Protocol-layer** multi-token coordination shapes *token hyperedges* and entropy metrics. The multi-scale nature of attacks, spanning local contract clusters to global market distortions, further justifies our *spectral-spatial-original hybrid feature integration* in Section IV-B.4.

IV. PROPOSED APPROACH

In this section, we present the comprehensive pipeline of our framework, covering data collection and preprocessing, feature extraction and model construction, and flash loan attack detection. The overall workflow is illustrated in Fig. 6 and Section IV-C.

A. Data Collection and Preprocessing

To address *Challenge 2* (Data imbalance due to limited flash loan attack samples), we deliberately construct datasets that reflect real-world imbalanced distributions. We first collect as many diverse flash loan attack cases as possible across multiple chains and attack types, then build our primary training and testing dataset (D1) with attack transactions comprising no more than 5% of the total. This design ensures that model evaluation occurs under realistic class-imbalanced conditions, providing a rigorous assessment of detection capability in genuine deployment scenarios.

1) *Data Collection*: We construct a comprehensive flash loan dataset comprising four components: flash loan *attack* data, *benign* transactions, *large-scale* Ethereum data, and a contract-platform mapping table.

Flash loan attack data: We collect attack cases from the REKT database [11], Leishen [5], and GitHub [12]. REKT is a professional and authoritative repository that systematically archives and analyzes security incidents, exploits, and vulnerabilities across DeFi protocols. To ensure authenticity, blockchain security experts cross-verify all samples with official incident reports, removing unverified cases. The final dataset covers seven

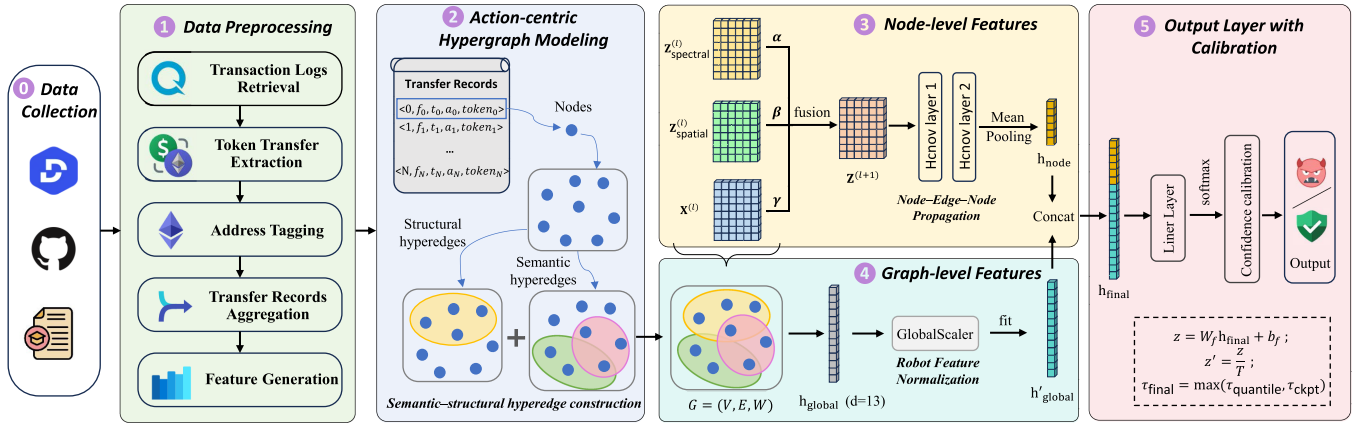


Fig. 6. The workflow of the proposed framework *FlashShield*, including data collection, data preprocessing, action-centric hypergraph modeling, node and graph level feature extraction, and a calibrated output layer for classifying attacks and benign transactions.

major chains, including Ethereum, BSC, Arbitrum, Avalanche, Base, Fantom, and Polygon.

Benign transactions: Benign samples come from (1) the dataset released by *Leishen* [5] and (2) filtered subsets of *DeFiScope* D3 from *DeFiScope* [6]. The *DeFiScope* D3 dataset extracted from *DeFort* D2 [7] was constructed by the authors of *DeFort* using verified DeFi applications from *DeFiLlama*, and further cross-checked with blockchain security databases and incident reports to ensure that no attack events occurred within the sampling period. Following *Leishen*, flash loan transactions are identified by function-call signatures (swap, uniswapV2Call, flashLoan). From *DeFiScope* D3's 96,800 benign DeFi transactions, we extract 576 valid flash loan records for consistent comparison.

Large-scale Ethereum transactions: We collect 12,102 flash loan transactions from block heights 17,800,000-23,080,000 (July 29, 2023-August 06, 2025) using QuickNode RPC [46]. Ethereum, with the largest DeFi activity and TVL ($\approx$ \$85 B) [47], provides rich real-world samples of attack scenarios with high value.

Contract-platform mapping: To deanonymize public DeFi contracts, we build a mapping table linking smart contract addresses to platform names. We aggregate data from Rayo [48], Alchemy [49], and Etherscan's *Label Cloud* [50], followed by deduplication and fuzzy matching, yielding 161,119 address-to-platform entries. This mapping prevents contracts owned by platforms (e.g., fee receivers, treasuries) from being misclassified as attackers.

These steps yield an integrated dataset covering both malicious and benign transactions, as well as platform metadata, forming the basis for later modeling and evaluation.

2) Data Preprocessing: We retrieve, label, and aggregate transaction information before modeling.

Step 1. Retrieve logs: Transaction receipts are obtained via the Alchemy API [49] using scripts that automatically collect detailed logs.

Step 2. Extract token transfers: From each log entry's `topics`, `data`, and `address` fields, we extract sender, receiver, amount, and token identifiers.

Step 3. Annotate contract addresses: Addresses found in the mapping table are labeled with their corresponding DeFi platforms; unlabeled addresses are considered private or unmatched contracts.

Step 4. Aggregate transfer records: Following [5], we remove redundant transfers within the same application. Furthermore, We merge consecutive transfers with the same *from* account or *to* account and nearly equal values.

Step 5. Compute global features: From the aggregated records, we compute 13 kinds of graph-level statistics: $(N_{nodes}, N_{addrs}, N_{tokens})$, $(\bar{a}_{rel}, a_{rel}^{max}, \sigma_{a,rel}, G_a)$, $(d_{in}^{max}, \bar{d}_{in}, d_{out}^{max}, \bar{d}_{out})$, (H_{token}, H_{addr}) . These features capture the scale, fund flow balance, and diversity of token and address interactions, serving as input for subsequent modeling.

B. Feature Representation and Model Design

To address *Challenge 3* (atomicity and behavioral complexity), we design a hypergraph-based detection framework that captures intricate transactional semantics and structural dependencies. *FlashShield* models each fund transfer action as a node, employs action-centric hypergraph modeling to represent complex fund flows, and constructs semantic-structural hyperedges that integrate semantic relations with structural similarity. The architecture further incorporates spectral-spatial-original hybrid feature integration to fuse global and local representations, and node-graph feature fusion to combine multi-level embeddings for robust anomaly detection.

Although transfer records are aggregated during preprocessing, a single flash loan transaction may still contain dozens to hundreds of transfer actions. To uncover the trader's intent within such dense transactional behaviors, we perform advanced feature extraction and modeling based on hypergraph. Flash loan attack detection is then achieved using Hypergraph Neural Networks (HGNNs) [51]. In the following, we first provide a *model architecture overview*, and then describe four core components: *action-centric hypergraph modeling*, *semantic-structural hyperedge construction*, *spectral-spatial-original hybrid feature integration*, and *multi-level feature embedding fusion*.

1) *Model Architecture Overview*: Our detection model follows a hierarchical architecture built upon the HGNN framework. Given a flash loan transaction represented as a hypergraph

$$G = (V, E, W), \quad (1)$$

where V denotes the set of token transfer actions (nodes) and E denotes the set of semantic-structural hyperedges, the model first encodes node features into latent embeddings and then performs multi-layer hypergraph convolutions to aggregate information across correlated actions.

Each hypergraph convolutional layer executes a node-edge-node message propagation process, formulated as:

$$X^{(l+1)} = \sigma \left(D_v^{-\frac{1}{2}} H W D_e^{-\frac{1}{2}} H^\top D_v^{-\frac{1}{2}} X^{(l)} \Theta^{(l)} \right), \quad (2)$$

where $H \in \{0, 1\}^{|V| \times |E|}$ is the incidence matrix encoding which nodes belong to which hyperedges, D_v and D_e are the diagonal matrices of vertex and hyperedge degrees, W is the hyperedge weight matrix, $\Theta^{(l)}$ denotes the learnable parameter of layer l , and $\sigma(\cdot)$ is a nonlinear activation function (ReLU in our implementation).

After two hypergraph convolutional layers, the node embeddings are aggregated to obtain a graph-level representation. The global transaction-level features extracted in Section IV-A are first normalized by a *GlobalScaler* module and then concatenated with the graph-level representation to form an integrated feature vector. The combined representation is subsequently fed into a classifier for attack detection, followed by a confidence calibration module to adjust the predicted probabilities.

2) *Action-centric Hypergraph Modeling*: Conventional blockchain transaction modeling usually represents entities such as user accounts, platforms, or smart contracts as nodes and connects them via transaction edges. However, this account-centric paradigm aggregates multiple transactions into the same node, making it difficult to explicitly model temporal dependencies and inter-transactional influences. Moreover, it focuses more on account-level characteristics rather than action-level features, thus being more suitable for scenarios such as account deanonymization rather than flash loan attack detection.

To address this limitation, we propose an *action-centric hypergraph modeling* approach: each token transfer action within a flash loan transaction is treated as a node, and the entire transaction is represented as a hypergraph. Formally, each node $v_i \in V$ corresponds to:

$$v_i = \langle \text{index}_i, \text{from}_i, \text{to}_i, \text{amount}_i, \text{token}_i \rangle. \quad (3)$$

The incidence matrix H encodes the membership relation:

$$H(v_i, e_j) = \begin{cases} 1, & v_i \in e_j, \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

Since all token transfer actions occur atomically within a single transaction, temporal order is encoded simply by their sequential indices.

3) *Semantic-Structural Hyperedge Construction*: Flash loan attacks often involve a series of correlated action patterns, such as repeated buy and sell cycles for price manipulation or multiple invocations of vulnerable contracts for asset theft. Hence, token

transfer actions exhibit both feature similarity and semantic dependency. To capture these relationships, we construct one type of structural hyperedges and four types of semantic hyperedges:

- *Structural hyperedges*: constructed based on K-nearest neighbors (KNN) in the node feature space.
- *Semantic hyperedges*: 1) *Address hyperedges*: connecting actions that share the same sender or receiver address. 2) *Token hyperedges*: linking actions involving the same token. 3) *Temporal hyperedges*: connecting temporally adjacent actions within a sliding window. 4) *Flow-chain hyperedges*: capturing fund flow continuity where the receiver of one action is the sender of the next.

Let $\mathbf{x}_i$ denote the feature vector of node v_i . The KNN-based structural hyperedges are defined using cosine similarity:

$$\text{sim}(v_i, v_j) = \frac{\mathbf{x}_i \cdot \mathbf{x}_j}{\|\mathbf{x}_i\| \|\mathbf{x}_j\|}. \quad (5)$$

Each node is connected to its top- K most similar nodes to form a *structural* hyperedge. For *semantic* hyperedges, we instantiate four types:

Address hyperedges (same sender/receiver):

$$\mathcal{E}_{\text{addr}} = \{ \{v_i, v_j\} \mid \text{from}_i = \text{from}_j \vee \text{to}_i = \text{to}_j \}. \quad (6)$$

Token hyperedges (same token):

$$\mathcal{E}_{\text{tok}} = \{ \{v_i, v_j\} \mid \text{token}_i = \text{token}_j \}. \quad (7)$$

Temporal-window hyperedges (window size w):

$$\mathcal{E}_{\text{tmp}} = \{ \{v_i, \dots, v_j\} \mid 1 \leq i < j \leq n, \ 2 \leq j - i + 1 \leq w \}. \quad (8)$$

Flow-chain hyperedges (via intermediate address a):

$$\begin{aligned} I(a) &= \{ k \mid \text{to}_k = a \text{ or } \text{from}_k = a \}, \\ \mathcal{E}_{\text{flow}} &= \{ \{v_k \mid k \in I(a)\} \mid |I(a)| \geq 2 \}. \end{aligned} \quad (9)$$

These structural and semantic hyperedges jointly capture spatial, semantic, and temporal dependencies within a transaction.

4) *Spectral-Spatial-Original Hybrid Feature Integration*: Each hypergraph convolutional layer aggregates information through both *spectral filtering* and *spatial neighborhood aggregation*, enabling the model to capture dependencies across multiple scales and of higher order. For node feature matrix $X^{(l)}$, the hybrid feature integration is formulated as:

$$Z^{(l+1)} = \alpha Z_{\text{spectral}}^{(l)} + \beta Z_{\text{spatial}}^{(l)} + \gamma X^{(l)}, \quad (10)$$

where α , β , and γ are learnable weights balancing spectral, spatial, and original representations. This mechanism allows simultaneous encoding of both global (spectral) and local (spatial) correlations among token transfer actions.

5) *Multi-level Feature Embedding Fusion*: After the final convolutional layer, we obtain fused node embeddings $\{z_i\}$. We aggregate them via mean pooling to derive the graph-level feature:

$$\mathbf{h}_{\text{node}} = \frac{1}{|V|} \sum_{v_i \in V} z_i. \quad (11)$$

The global transaction feature $\mathbf{h}_{\text{global}}$ obtained from Section IV-A is then concatenated with the node-level representation to form the final transaction embedding:

$$\mathbf{h}_{\text{final}} = [\mathbf{h}_{\text{node}} \parallel \mathbf{h}_{\text{global}}]. \quad (12)$$

The combined embedding is first projected into the classification space through a linear transformation layer:

$$\mathbf{z} = W_f \mathbf{h}_{\text{final}} + b_f, \quad (13)$$

where $W_f \in \mathbb{R}^{(64+d_g) \times 2}$ and b_f are trainable parameters of the linear layer. To calibrate prediction confidence, the logits are scaled by a temperature parameter $T > 0$:

$$\mathbf{z}' = \frac{\mathbf{z}}{T}. \quad (14)$$

The calibrated logits $\mathbf{z}'$ are then passed through a softmax function to obtain class probabilities:

$$\mathbf{p} = \text{Softmax}(\mathbf{z}'). \quad (15)$$

A dynamic threshold $\tau_{\text{final}} = \max(\tau_{\text{quantile}}, \tau_{\text{ckpt}})$ is subsequently applied to $\mathbf{p}$ for final decision making:

$$\hat{y} = \begin{cases} 1, & \text{if } p_{\text{attack}} \geq \tau_{\text{final}}, \\ 0, & \text{otherwise.} \end{cases} \quad (16)$$

This confidence calibration mechanism ensures that the model's predicted probabilities better align with its true reliability, providing more trustworthy detection results. Specifically, τ_{ckpt} is obtained at training time as the threshold maximizing F1 on the validation set subject to $\text{fp_rate} \leq 0.003$, and saved with the checkpoint. τ_{quantile} is an optional inference-time limiter set to the $(1 - \alpha)$ -th percentile of the score distribution (default $\alpha = 0.025$) to cap alert volume in high-throughput deployment; it is disabled by default. Taking their maximum ensures the alert-rate constraint can only tighten, never relax, the fp-rate guarantee.

C. Detection Pipeline

Building upon data preprocessing and hypergraph representation, we design a comprehensive detection pipeline, as illustrated in Fig. 6. The workflow proceeds through five stages: *data preprocessing*, *action-centric hypergraph modeling*, *node-level feature propagation*, *graph-level feature fusion*, and *output with confidence calibration*.

Step 1. Data Preprocessing: For each flash loan transaction, we retrieve token transfer logs, tag contract addresses, and aggregate transfer records. Each transfer action is represented as $\langle \text{index}, \text{from}, \text{to}, \text{amount}, \text{token} \rangle$. From these, we generate a global transaction feature vector $\mathbf{h}_{\text{global}}$ with 13 dimensions that characterizes each transaction in terms of token diversity, address activity, and fund-flow structure.

Step 2. Action-centric Hypergraph Modeling: Each transfer action becomes a node v_i in the hypergraph $G = (V, E, W)$. We construct one structural hyperedge and four semantic hyperedges: address, token, temporal, and flow-chain, which jointly capture structural proximity, co-occurrence, and temporal dependencies among actions.

TABLE III
THE BENCHMARK DATASETS USED FOR EVALUATION

Dataset	Information	Source	RQs
D1	153 real-world flash loan attack transactions and 2907 benign flash loan transactions	REKT, Leishen, GitHub	RQ1, RQ3
D2	576 benign flash loan transactions	DeFiScope	RQ2
D3	12,102 in-the-wild flash loan transactions	Ethereum	RQ4

Step 3. Node-level Feature Propagation: Two hypergraph convolutional layers perform *Node-Edge-Node Propagation*, integrating spectral, spatial, and original features weighted by learnable coefficients $(\alpha, \beta, \gamma) = (0.3, 0.3, 0.4)$. This fusion produces refined node embeddings and the node embeddings are then mean-pooled into $\mathbf{h}_{\text{node}}$.

Step 4. Graph-level Feature Scaling and Fusion: The node-level feature $\mathbf{h}_{\text{node}}$ is concatenated with the scaled global feature $\mathbf{h}'_{\text{global}} = \text{GlobalScaler}(\mathbf{h}_{\text{global}})$ to form the integrated representation $\mathbf{h}_{\text{final}} = [\mathbf{h}_{\text{node}} \parallel \mathbf{h}'_{\text{global}}]$.

Step 5. Output with Confidence Calibration: The integrated representation $\mathbf{h}_{\text{final}}$ is first projected to logits $\mathbf{z} = W \mathbf{h}_{\text{final}} + b$, which are calibrated through temperature scaling $\mathbf{z}' = \mathbf{z}/T$ and then converted into probabilities $\mathbf{p} = \text{Softmax}(\mathbf{z}')$. A dynamic threshold $\tau_{\text{final}} = \max(\tau_{\text{quantile}}, \tau_{\text{ckpt}})$ is finally applied to control decision confidence, where a transaction is classified as an attack if $p_{\text{attack}} \geq \tau_{\text{final}}$.

The system supports both offline batch evaluation on historical data and real-time on-chain detection via QuickNode Stream API [46].

V. EVALUATION

In this section, we evaluate the proposed *FlashShield* framework. Specifically, the experiments are designed to answer the following research questions (RQs):

- **RQ1:** How effective is *FlashShield* in detecting flash loan attacks compared with state-of-the-art methods?
- **RQ2:** How well does *FlashShield* control false positives on benign flash loan transactions?
- **RQ3:** How do different modules affect the overall performance of *FlashShield*?
- **RQ4:** How does the proposed approach perform on large-scale real-world data?

A. Experimental Setup

(1) Datasets: To address the above four RQs, we divide the collected data (Section IV-A) into three datasets. As shown in Table III, the first dataset, *D1*, contains 153 real-world flash loan attack transactions and 2,907 benign flash loan transactions, and is used to evaluate RQ1 and RQ3. We follow the dataset construction strategy proposed in previous work [24], which randomly mixes benign transactions with attack transactions while ensuring that the attack ratio does not exceed 5%. The second dataset, *D2*, includes 576 benign flash loan transactions and is used to evaluate RQ2. The third dataset, *D3*, consists of

12,102 flash loan transactions collected from Ethereum and is used to evaluate RQ4. All transactions in D3 that overlap with attack data in D1 are removed to prevent data leakage.

(2) *Baselines*: We compared *FlashShield* with four state-of-the-art approaches: *Leishen* [5], *DeFort* [7], *DeFiScope* [6], and *MetaSuites* [52].

- *Leishen* detects flash loan attacks through manually defined rules and pattern matching.
- *DeFort* builds a price manipulation model by monitoring token prices and profit calculation mechanisms.
- *DeFiScope* leverages large language models (LLMs) to abstract price computation logic from smart contract code, reconstructs high-level DeFi semantics via graph-based analysis, and summarizes eight price manipulation patterns for detection.
- *MetaSuites*, developed by the blockchain security company *BlockSec* [53], is a ChatGPT-based blockchain transaction analysis platform. It utilizes blockchain explorer data (e.g., Etherscan) to interpret transaction intents and integrates more than 300 million labeled addresses.

We re-implemented *Leishen* from its released core code and optimized it to retrieve data via RPC services, eliminating the need to rely on storage-intensive full nodes of blockchain. Since *DeFort*'s implementation is not publicly available, we reached out to its authors and conducted tests on our dataset in their local environment. While *DeFiScope* has open-sourced its code, our testing revealed API tool inconsistencies that led to erroneous results. As a result, we based our evaluation on the published data and reported results. Since *MetaSuites* is a browser extension, we manually reviewed each transaction during testing, including its detection results and the labeling of both parties' addresses.

(3) *Implementation and Parameter Settings*: We implemented the data preprocessing module in Go and the neural network module in Python, linking them via custom scripts. All experiments were conducted on a desktop machine running Ubuntu 22.04, equipped with an Intel(R) Core(TM) i7-14700KF CPU (20 cores), an NVIDIA GeForce RTX 4060 Ti GPU, and 32 GB of RAM. We used the Adam optimizer with an initial learning rate of 1×10^{-4} , and applied a StepLR decay by a factor of 0.5 every 5 epochs. The model was trained for 15 epochs with a batch size of 1, since each transaction graph was processed individually. Alert-rate limiting (τ_{quantile}) is disabled by default in all experiments; it can be enabled for high-throughput deployment to bound the total alert volume. The detailed parameter settings are illustrated in Table IV.

B. RQ1: Comparison Experiment

In this section, we conduct comparative experiments to address RQ1. Dataset *D1* is randomly divided into training and testing sets with an 8:2 ratio, where the test set contains 31 flash loan attack transactions and 589 benign flash loan transactions. A random split is adopted because *D1* contains only 153 labeled attack transactions in total, making a direct chronological split statistically unreliable for evaluation. Temporal generalization is instead assessed through the large-scale *D3* experiment in

TABLE IV
CORE EXPERIMENTAL PARAMETERS

Parameter	Value / Description
Random Seed	42
Integration Weights (α, β, γ)	Initialized as (0.3, 0.3, 0.4), learnable.
Hidden Dimensions	256 (hidden) $\rightarrow$ 64 (embedding)
Regularization	Dropout ($p = 0.5$) applied to hidden layers
Optimizer	Adam
Learning Rate	1×10^{-4}
Training Epochs	15
Class-Balanced Weighting	$\beta = 0.9999$
Loss Function	Focal Loss ($\gamma = 2.0$)
Threshold Search	metric = "fp_rate", target = 0.003

Section V-E, which evaluates *FlashShield* on 12,102 in-the-wild transactions collected over a roughly two-year window entirely unseen during training. As a supplementary experiment, we also applied a chronological split to the full transaction pool (training: Feb. 2020–Jul. 2021; test: Jul. 2021–Jul. 2024), yielding a training attack ratio of only 1.5% (37 samples) versus 16.1% in the test set (98 samples) due to the temporal distribution of labeled data. Under this setting, *FlashShield* achieves a precision of 91.2% (5 FP out of 511 benign test transactions), confirming a low false-alarm rate under deployment-like conditions; the recall of 53.1% primarily reflects the severe training-set class imbalance and the natural temporal evolution of attack strategies after the training cutoff. Following the comparative setup in [6], [7], we evaluate the detection performance of *FlashShield* against *Leishen* [5], *DeFort* [7], *DeFiScope* [6], and *MetaSuites* [52].

To provide a detailed analysis, we collect and display for each attack transaction the event name, timestamp, loss amount, and blockchain network information. Furthermore, we categorize each transaction by its corresponding flash loan attack type. A detailed comparison of detection results is summarized in Table V.

As shown in Table V, *FlashShield* successfully identified 30 out of 31 attacks, achieving a recall rate of 96.7%. *MetaSuites* detected 21 attacks with a recall rate of 67.7% (29% improvement). It is worth noting that we adopt a relatively lenient evaluation rule for *MetaSuites*: since it benefits from a database of over 300 million labeled addresses, if a transaction is not explicitly classified as an attack but one of its participating addresses is labeled as an *exploiter*, we consider it as a detected case. *Leishen* and *DeFort* both detected 9 attacks (recall 29.0%), while *DeFiScope* detected 5 attacks.

Based on these findings, we further analyze the number of detected attacks across different flash loan attack types, as shown in Table VI. The results indicate that *Leishen*, *DeFort*, and *DeFiScope* perform best on *price manipulation* (PM) flash loan attacks, but their recall drops markedly for the other types. This gap arises because these methods were originally designed with price manipulation as the primary target and largely overlook the broader diversity of flash loan attack types.

Next, we analyze the single missed attack case [54] by *FlashShield*. This transaction corresponds to the *Rari Capital* attack event (May 8, 2021). According to the official incident report, the attack occurred in the lending pool integrated between *Rari Capital* and *Alpha Finance*. After obtaining liquidity via

TABLE V

DeFi INCIDENTS DETECTION RESULTS (✓ = DETECTED; LS: LEISHEN, DF: DEFORT, DS: DEFISCOPE, MS: METASUITES, FS: FLASHSHIELD). DIFFERENT ROWS REPRESENT DIFFERENT ATTACK TRANSACTIONS, WHICH MAY COLLECTIVELY FORM A SINGLE SECURITY EVENT. AS A RESULT, THE "TITLE" COLUMN MAY BE THE SAME, BUT THE DETECTION RESULTS FROM DIFFERENT TOOLS CAN VARY, AS SEEN IN THE RARI CAPITAL INCIDENT.

#	Title	Time (UTC)	Loss (\$)	Cat.	Chain	LS	DF	DS	MS	FS
1	Earning Farm	2022/10/14	1.36M	BLF	ETH				✓	✓
2	UN Token	2023/06/06	26.6K	CVE	BSC			✓	✓	✓
3	Paraluni	2022/03/13	1.66M	CVE	BSC				✓	✓
4	EtnProduct	2022/08/04	10.5K	BLF	BSC					✓
5	Sumer.Money	2024/04/12	350K	CPMD	BASE					✓
6	JulSwap	2021/05/29	1.5M	PM	BSC		✓		✓	✓
7	DualPools	2024/02/15	41.3K	CVE	BSC				✓	✓
8	Zabu Finance	2021/09/11	3.2M	BLF	AVAXC					✓
9	Beosin Alert	2022/07/25	45K	CVE	BSC		✓			✓
10	OMNI	2022/07/10	1.43M	CVE	ETH				✓	✓
11	yearn.finance (2)	2023/04/13	11.5M	CPMD	ETH		✓			✓
12	FEG Token(2)	2022/05/16	1.9M	CVE	ETH				✓	✓
13	BRA Token	2023/01/10	225K	BLF	BSC				✓	✓
14	AutoShark	2021/10/01	581K	PM	BSC					✓
15	Smart Bank Token	2024/07/11	56K	CVE	BSC				✓	✓
16	Zunami Protocol	2023/08/13	2.18M	PM	ETH			✓	✓	✓
17	Rari Capital	2021/05/08	15M	CPMD	ETH	✓			✓	✓
18	FEG Token(1)	2022/05/15	1.3M	CVE	BSC		✓		✓	✓
19	Saddle Finance	2022/04/30	11M	PM	ETH	✓			✓	✓
20	JulSwap	2021/05/27	1.5M	PM	BSC	✓	✓		✓	✓
21	Belt Finance	2021/05/29	6.3M	BLF	BSC	✓		✓		✓
22	Midas Capital	2023/01/15	650K	CVE	POLYGON		✓		✓	✓
23	2omb	2022/04/18	1.6M	BLF	FTM					✓
24	Radiant Capital	2024/01/02	4.34M	BLF	ARBTRUM	✓			✓	✓
25	Grim Finance	2021/12/19	40M	CVE	FTM					✓
26	EVODeFi	2021/06/10	1.74M	BLF	BSC					✓
27	Value DeFi	2020/11/14	7M	PM	ETH	✓	✓	✓	✓	✓
28	bzx-2	2020/02/18	666K	PM	ETH	✓	✓	✓	✓	✓
29	Rari Fuse	2022/04/30	79.7M	CVE	ETH				✓	✓
30	xToken	2021/05/12	24M	PM	ETH	✓			✓	✓
31	Harvest Finance	2020/10/26	33.8M	PM	ETH	✓	✓		✓	✓

TABLE VI

DETECTION RESULTS OF ALL METHODS FOR DIFFERENT TYPES OF FLASH LOAN ATTACKS

Attack Type	Leishen	DeFort	DeFiScope	MetaSuites	FlashShield	Total
PM	6	5	3	8	9	9
BLF	2	0	1	3	8	8
CVE	0	3	1	9	11	11
CPMD	1	1	0	1	2	3
Total	9	9	5	21	30	31

a flash loan, the attacker exploited inconsistencies in cross-protocol valuation by manipulating the `totalETH()` return value in Alpha's `ibETH` contract. This manipulation distorted Rari's internal asset ratio calculation, allowing the attacker to repeatedly deposit and withdraw funds in a short period, ultimately extracting around \$11 million in assets. This incident exemplifies a *Cross-Protocol Mechanism Dislocation (CPMD)* attack, where mismatched valuation or state-synchronization mechanisms among different DeFi protocols introduce systemic risk.

Using the *Phalcon Explorer* [55], we further inspect the transaction's fund flow and balance changes. We observe that the majority of the transferred funds circulated internally between the Rari and `ibETH` contracts, with minimal external token movements. Therefore, we infer two main reasons why *FlashShield* failed to detect this case: **(1) Missing semantic**

coverage: The current semantic hyperedge construction does not capture cross-protocol valuation dependencies or call-chain interactions, which limits the model's ability to detect anomalies arising from protocol interface mismatches. Specifically, the missing relation can be characterized as a *cross-protocol state-read dependency*: a link between actions that share an implicit economic coupling through inter-contract function calls (e.g., a price or balance read that propagates a manipulated state across protocol boundaries), rather than through explicit token transfers. Because *FlashShield*'s hyperedges are constructed over ERC-20 transfer actions, this silent valuation dependency is structurally invisible to the current representation. **(2) Weak explicit features:** Since most fund transfers occurred within integrated protocols, the aggregated token transfer sequence was relatively short, resulting in low model confidence. To enhance *FlashShield*'s capability in identifying CPMD-type attacks, future work can incorporate cross-protocol relational features that bridge the economic and semantic layers of DeFi interactions.

In addition to attack detection, we also examine the models' ability to correctly identify benign transactions contained in *D1*. *FlashShield* produces zero false positive, and both *Leishen* and *DeFort* report zero false positives on this subset. This result shows that *FlashShield* maintains a low false-positive rate even on mixed datasets containing both attack and benign flash loan transactions.

TABLE VII
FUSION OF GRAPH-LEVEL FEATURE ABLATION (%)

Model	Accuracy	F1	Precision	Recall
M0 (Full)	99.84	98.36	100.00	96.77
M1 (w/o Size-related)	-0.32	-3.28	-3.33	-3.22
M2 (w/o Amount-related)	-4.68	-86.60	-33.33	-90.32
M3 (w/o Degree-related)	-0.32	-3.28	-3.33	-3.22
M4 (w/o Entropy-related)	-3.71	-35.86	-39.39	-32.25
M5 (w/o ALL Graph-level Features)	-0.32	-3.44	0.00	-6.45

C. RQ2: False Positive Analysis

To address RQ2 and more thoroughly assess false positives beyond the mixed dataset in RQ1, we evaluate *FlashShield* on a separate benign-only dataset (*D2*) comprising 576 benign flash loan transactions. We compare it with *DeFort* and *DeFiScope*, which report zero false positives, while *FlashShield* produces only two. Using *Phalcon Explorer* [55], we find that both cases exhibit nearly identical fund-flow graphs with minor value differences; thus, we analyze one representative case in detail below.

(1) *Transaction information*: The FP transaction [56] was initiated from an MEXC hot wallet and invoked the *PancakeSwap Router v2* contract to perform a standard liquidity provision process: (1) swap *BabyDoge* tokens for *BNB/WBNB* using `swapExactTokensForETHSupportingFeeOnTransferTokens()` (2) call `addLiquidityETH()` to add liquidity to the *Cake-LP* pool, and (3) receive LP tokens transferred to a *GnosisSafeProxy* address. The fund-flow path *MEXC* → *PancakeRouter* → *Cake-LP* → *GnosisSafeProxy* shows normal liquidity operations without cyclic transfers, profit loops, or reentrancy, indicating a benign liquidity-adding transaction.

(2) *Data analysis after preprocessing*: After aggregation, this transaction yielded 7 nodes, each representing a token transfer action. Feature inspection shows: (1) multiple tokens (*BabyDoge*, *BNB/WBNB*, *Cake-LP*); (2) three transfers involving *BabyDoge* with alternating directions and similar amounts; (3) two symmetric transfers of *BNB/WBNB* showing a *round-trip* pattern. The resulting graph exhibits dense token exchange loops and partial fund recirculation.

(3) *Cause of false positive*: The false alarms are mainly due to (i) *amplified structural similarity* and (ii) *missing semantic constraints*:

- *Structural overfitting*: Frequent token swaps between *BabyDoge* and *WBNB/BNB* create dense KNN clusters resembling price manipulation subgraphs in the training set, causing HGNN's spectral-spatial aggregation to raise confidence scores.
- *Lack of function semantics*: Node features lack indicators of function calls such as `addLiquidityETH()` or protocol tags like *PancakeSwap*, making it hard for the model to distinguish normal AMM interactions from manipulative loops.

Both false-positive cases exhibit identical *PancakeSwap* liquidity-adding semantics and similar multi-token flow patterns, indicating that *FlashShield* may misclassify complex but legitimate AMM operations as price manipulation attacks

when relying solely on structural and fund-flow features. To mitigate this, we plan to (i) incorporate function and protocol semantics into action nodes and hyperedges, and (ii) expand benign samples of liquidity operations. These refinements are expected to reduce false positives while maintaining recall. Although *FlashShield*'s false-positive rate is slightly higher than that of rule-based baselines (*DeFort* and *DeFiScope*), its broader detection coverage and adaptability demonstrate strong potential for large-scale, real-time on-chain monitoring.

D. RQ3: Ablation Study

In this section, we conduct ablation experiments to answer RQ3. We evaluate the contribution of the main components of *FlashShield*, including: (i) graph-level features, (ii) spectral-spatial-original hybrid feature integration, (iii) semantic-structural hyperedge construction, and (iv) confidence calibration mechanism.

(1) *Graph-level feature ablation*: The 13 kinds of global graph features are grouped into six configurations:

- *M0 (Full)*: the complete model with node-level features and all 13 kinds of graph-level features.
- *M1 (w/o Size-related Features)*: remove three kinds of size-related graph-level features: $(N_{nodes}, N_{addrs}, N_{tokens})$.
- *M2 (w/o Amount-related Features)*: remove four kinds of amount-related graph-level features: $(\bar{a}_{rel}, a_{rel}^{max}, \sigma_{a,rel}, G_a)$.
- *M3 (w/o Degree-related Features)*: remove four kinds of degree-related graph-level statistics: $(d_{in}^{max}, \bar{d}_{in}, d_{out}^{max}, \bar{d}_{out})$.
- *M4 (w/o Entropy-related Features)*: remove two kinds of entropy-related graph-level features: (H_{token}, H_{addr}) .
- *M5 (w/o ALL Graph-level Features)*: remove all 13 kinds of graph-level features.

As shown in Table VII, removing either the amount-related or entropy-related global features (M2, M4) causes the largest performance degradation, especially in F1 and recall. This indicates that the global distribution of transaction amounts, tokens, and addresses is crucial for distinguishing attacks, which typically involve frequent, high-value transfers and interactions across multiple addresses.

When all global feature types are removed simultaneously (M5), the recall still drops markedly compared to the full model, confirming that graph-level global features as a whole play an important role in exposing flash loan attacks.

More interestingly, ablating only the amount-related or only the entropy-related features (M2, M4) degrades performance even more than removing all global features (M5). This suggests a strong complementarity between these two groups of features: amount-related features capture the global distribution and concentration of monetary flows, while entropy-related features characterize the diversity and dispersion of addresses and token types. In flash loan attacks, large, short-lived bursts of capital tend to co-occur with complex, multi-address and multi-token interaction patterns. Dropping either feature group breaks this joint signal and can even mislead the model, whereas using them together provides a more complete global view of

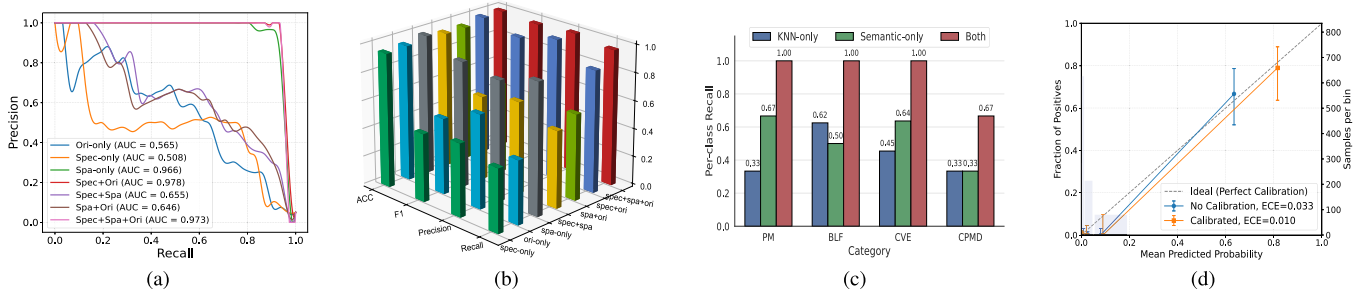


Fig. 7. (a) PR curves for feature integration strategies; (b) Accuracy, F1, precision, and recall of feature integration results; (c) Impact of hyperedge construction strategies on the recall rate of each type of attack; (d) Reliability diagram.

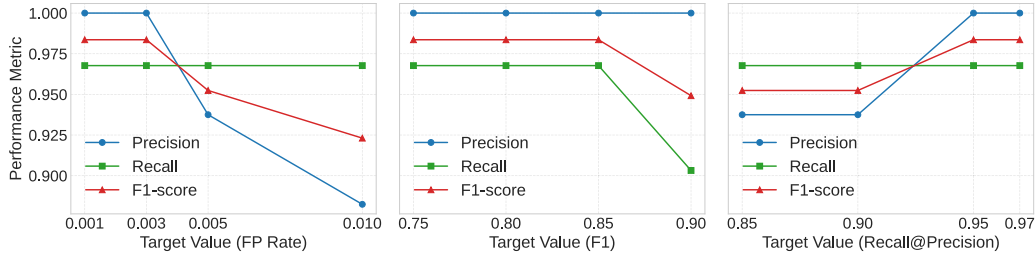


Fig. 8. Effect of different target values under different threshold selection criteria (FP rate, F1, and Recall@Precision) on Precision, Recall, and F1 scores.

malicious transaction behaviors and thus yields substantially better detection performance.

(2) *Feature integration strategy (Spec, Spa, Ori)*: We test seven combinations: three single components (Spec-only, Spa-only, Ori-only), three pairwise integrations (Spec+Spa, Spec+Ori, Spa+Ori), and the full combination (Spec+Spa+Ori). Fig. 7(a) shows the PR curves of all integration variants. Spec+Ori and the full model achieve comparable results, while Spec-only (AUC=0.508) and Ori-only (AUC=0.565) perform worst, confirming that proper feature integration significantly enhances detection performance. Fig. 7(b) illustrates Precision, Recall, F1, and Accuracy for all variants. All models achieve high accuracy, but Spa-only shows higher recall, explaining why the full model’s most notable improvement over Spec+Ori lies in recall. This highlights the importance of spatial features in capturing local interaction patterns crucial for attack detection.

(3) *Hyperedge construction (KNN vs Semantic)*: We compare three settings: KNN-only, Semantic-only, and KNN+Semantic. As shown in Fig. 7(c), KNN-only performs worst on price manipulation (PM) but best on Business Logic Flaw (BLF) attacks, likely due to their stronger structural regularities. Semantic-only performs best on PM attacks, since all four semantic hyperedges are fund-flow-related, matching the characteristic *large-injection-price-disruption-large-withdrawal* pattern.

(4) *Calibration mechanism*: We compare models with and without calibration (temperature scaling + threshold selection). Fig. 7(d) shows that before calibration (blue), predictions are slightly overconfident at mid-high probability regions, while after calibration (orange), the reliability curve aligns closely with the ideal diagonal. The expected calibration error (ECE) decreases from 0.033 to 0.010, and both Brier score and NLL improve significantly, demonstrating that temperature scaling effectively enhances probability reliability. Fig. 8 further

presents how different target values under FP rate, F1, and Recall@Precision criteria affect Precision, Recall, and F1. Due to data imbalance, a smaller FP-rate target yields better balance, whereas overly high targets in F1 degrade performance. For Recall@Precision, a higher target helps maximize recall while maintaining acceptable precision.

E. RQ4: Large-Scale Real-World Evaluation

To answer RQ4, we conduct large-scale experiments on 12,102 flash loan transactions of (D3). When performing offline batch detection on this large-scale dataset, *FlashShield* takes 113.334 seconds in total, corresponding to an average of 0.00936 seconds per transaction. Moreover, when we deploy *FlashShield* in a streaming setting using a pipeline with QuickNode’s Stream service [46], it achieves an average processing time of 2.39 seconds per transaction, which is lower than the block generation time of both Ethereum (12 seconds) and BNB Chain (3 seconds). The system identified 109 suspicious transactions, which were subsequently examined by DeFi security experts.

Experts evaluated each case using various tools, including blockchain explorers, search engines, GPT-5, and existing detection systems. The evaluation considered fund-flow paths, function call sequences, balance changes, and disclosure reports.

The classification follows a three-dimensional framework. (1) *Intent dimension*. Transactions are classified by underlying intent: **malicious** intent to steal or exploit, **deceptive** intent for phishing, and **legitimate** intent for economic operations. (2) *Identification source*. Classification relies on verifiable evidence: public disclosure verification, address labels from trusted explorers, and transaction patterns from fund-flow analysis. (3) *Behavioral features*. Each category is characterized by

TABLE VIII
MANUAL ANALYSIS RESULTS OF LARGE-SCALE REAL-WORLD EVALUATION

Event Category	Counterparty Tag / Key Behavioral Feature	Description	Is Attack?	Sample Count	Percentage (%)
Known Incidents	UwU Lend Incident, KyberSwap Incident, FloorDAO Incident	Publicly disclosed exploit incidents	✓	5	4.59
Exploiting	Equilibria Exploiter, Exploiter, Gain Token Exploiter, Grok Token Exploiter, uniBTC Exploiter	Exploiters conducting exploits or withdrawing funds	✓	18	16.51
Address poisoning	Simple actions, extremely low transaction amounts	A novel type of address poisoning	†	25	22.94
Arbitrage	MEV Transaction EigenPhi, MEV Transaction Flashbots	MEV-related activities (arbitrage/MEV extraction); non-malicious but highly automated	✗	4	3.67
	dYdX, UNI-V2; Uniswap V3; DAI, UNI-V2	Common DEX platform trades / liquidity operations	✗	27	24.77
Others (normal)	dYdX, ProtocolFeeRecipient, GnosisSafeProxy	Liquidity operations via GnosisSafeProxy	✗	19	17.43
	ChainTools: Yield Vault	Automated compounding / position maintenance in yield vaults	✗	5	4.59
	Mining, Burning, Staking, others	Other non-attack transactions	✗	6	5.50

† Suspected malicious activity (address poisoning pattern); classified separately from confirmed flash loan exploits.

discriminative indicators including fund-flow topology, transaction scale, and operational semantics.

Based on the above classification framework, we categorized all suspicious transactions into the following five classes: *Known Incidents*, *Exploiting*, *Address Poisoning*, *Arbitrage*, and *Others (normal)*. The distribution of results is shown in Table VIII. *Known Incidents* and *Exploiting* constitute confirmed attacks (23 transactions); *Address Poisoning* represents suspected malicious activities that match known address poisoning patterns [23] but are distinct from flash loan exploits in attack intent. Together, these three categories account for 44% of flagged transactions, comprising 43 previously unknown cases: 18 confirmed flash loan-related exploits (*Exploiting*) and 25 suspected address poisoning incidents.

(1) *Known Incidents*: These transactions, identified through manual inspection, are part of publicly disclosed security incidents, such as the UwU Lend, KyberSwap, and FloorDAO cases. Some of these incidents involve multiple attack transactions, with a total of five known attack transactions.

(2) *Exploiting*: Transactions in this category were initiated by addresses labeled as *Exploiter* by Phalcon Explorer. Analysis shows that they represent post-attack asset withdrawals (Step 7 in Fig. 1). A typical case exhibits a *flash-loan borrow* → *token routing* → *repayment* → *next loop* pattern, converting exploited assets through multi-pool swaps into main tokens (e.g., WETH), consistent with cash-out behaviors.

(3) *Address Poisoning*: These are small-value transfers matching the pattern described in [23]. They feature minimal WETH transfers (around 10^{-8} WETH) with no real profit or loss, created intentionally to leave transaction traces for phishing or address deception.

(4) *Arbitrage*: Normal arbitrage activities, including both MEV-based and DEX-based strategies. One typical *MEV Transaction EigenPhi* case involves multi-pool swaps and tax/burn tokens in a closed *flash-loan borrow-exchange-repay*

loop. Another, labeled *dYdX + UNI-V2* is a standard arbitrage attempt across dYdX and Uniswap V2, showing no exploit behavior. These transactions share several structural fingerprints with flash loan attacks, including multi-hop token transfers within a single atomic transaction, rapid asset inflow-outflow across multiple liquidity pools, and large price deviations on DEXs. Consequently, the model assigns elevated anomaly scores to these operations despite their legitimate economic intent.

(5) *Others (normal)*: These include GnosisSafeProxy-based liquidity operations, automated yield-vault compounding (e.g., *ChainTools: Yield Vault*), and mining-related transactions. For example, a transaction labeled *dYdX, ProtocolFeeRecipient, GnosisSafeProxy* reflects margin settlement through dYdX's flash-loan functions, while another, tagged *ChainTools: Yield Vault*, represents a typical auto-compounding workflow involving harvesting, rebalancing, and reinjecting liquidity within internal vault contracts. The false positives in this category stem from complex liquidity management operations through proxy or yield tools whose transfer-count density and hyperedge connectivity profile closely resemble those of confirmed attacks, leading the model to assign high initial suspicion scores.

Overall, *FlashShield* successfully identifies diverse on-chain behaviors, including unreported exploits and emerging attack types. The false positives in the Arbitrage and Others categories arise because *FlashShield*'s action-centric hypergraph captures transfer topology and temporal ordering rather than operational semantics, causing structurally similar but legitimate operations to receive elevated anomaly scores. Despite these false positives, the system demonstrates strong robustness and generalization in large-scale real-world monitoring.

VI. CONCLUSION

In this paper, we present *FlashShield*, a general detection framework for flash loan attacks based on Hypergraph Neural

Networks (HGNNs). By introducing action-centric hypergraph modeling and semantic-structural hyperedge construction, *FlashShield* captures complex transactional semantics and structural dependencies within flash loan transactions. The proposed spectral-spatial-original hybrid feature integration and node-graph Feature Fusion modules further enable hierarchical representation learning, allowing the system to accurately identify diverse and evolving attack patterns. Experimental results show that *FlashShield* boosts recall by 29% over state-of-the-art methods and uncovers 43 previously unknown malicious or suspicious activities (18 confirmed flash loan-related exploits and 25 suspected address poisoning incidents), demonstrating its robustness and potential for large-scale attack detection. For future work, we plan to integrate detection at the transaction granularity with code analysis at the smart contract granularity to enable automated vulnerability tracing and localization.

REFERENCES

- [1] Z. Zheng, S. Xie, H.-N. Dai, X. Chen, and H. Wang, "Blockchain challenges and opportunities: A survey," *Int. J. web grid Serv.*, vol. 14, no. 4, pp. 352–375, 2018.
- [2] SlowMist, "Slowmist hacked statistics," 2025, Accessed: Oct. 25, 2025. [Online]. Available: <https://hacked.slowmist.io/statistics/?c=all&d=all>
- [3] C. Gao, J. Ye, W. Yang, and Y. Xue, "Flash loan attack is more than just price oracle manipulation: A comprehensive empirical study," in *Proc. 25th Int. Conf. Softw. Qual. Rel. Secur.*, 2025, pp. 691–702.
- [4] M. Bartoletti, J. H.-Y. Chiang, and A. L. Lafuente, "SoK: Lending pools in decentralized finance," in *Proc. Int. Conf. Financial Cryptogr. Data Secur.*, 2021, pp. 553–578.
- [5] Q. Xia et al., "Detecting flash loan based attacks in ethereum," in *Proc. IEEE 43rd Int. Conf. Distrib. Comput. Syst.*, 2023, pp. 154–165.
- [6] J. Zhong et al., "Detecting various DeFi price manipulations with LLM reasoning," in *Proc. 40th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2025, pp. 1781–1793.
- [7] M. Xie et al., "DeFort: Automatic detection and analysis of price manipulation attacks in defi applications," in *Proc. 33rd ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2024, pp. 402–414.
- [8] Q. Kong, J. Chen, Y. Wang, Z. Jiang, and Z. Zheng, "Defitainter: Detecting price manipulation vulnerabilities in defi protocols," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2023, pp. 1144–1156.
- [9] Z. Chen, S. M. Beillahi, and F. Long, "FlashSyn: Flash loan attack synthesis via counter example driven approximation," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, 2024, pp. 1–13.
- [10] L. Zhou et al., "SoK: Decentralized finance (DeFi) attacks," in *Proc. IEEE Symp. Secur. Privacy*, 2023, pp. 2444–2461.
- [11] De.Fi, "Rekt Database: DeFi hacks, exploits and scams archive," 2025, Accessed: Oct. 25, 2025. [Online]. Available: <https://de.fi/rekt-database>.
- [12] Q. P. Kong, "Flash loan attack dataset," 2025, Accessed: Oct. 25, 2025. [Online]. Available: <https://github.com/kongqp/flash-loan-attack-dataset>
- [13] V. Buterin et al., "Ethereum white paper," *GitHub Repository*, vol. 1, no. 22–23, pp. 5–7, 2013.
- [14] F. Schär, "Decentralized Finance: On blockchain-and smart contract-based financial markets," *Review*, vol. 103, no. 2, 2021, doi: [10.20955/r.103.153-74](https://doi.org/10.20955/r.103.153-74).
- [15] S. Werner, D. Perez, L. Gudgeon, A. Klages-Mundt, D. Harz, and W. Knotenbelt, "SoK: Decentralized finance (DeFi)," in *Proc. 4th ACM Conf. Adv. Financial Technol.*, 2022, pp. 30–46.
- [16] V. Mohan, "Automated market makers and decentralized exchanges: A defi primer," *Financial Innov.*, vol. 8, no. 1, 2022, Art. no. 20.
- [17] K. Qin, L. Zhou, B. Livshits, and A. Gervais, "Attacking the DeFi ecosystem with flash loans for fun and profit," in *Proc. Int. Conf. Financial cryptogr. Data Secur.*, 2021, pp. 3–32.
- [18] J. Zhou, C. Hu, J. Chi, J. Wu, M. Shen, and Q. Xuan, "Behavior-aware account de-anonymization on ethereum interaction graph," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 3433–3448, 2022.
- [19] S. Miao et al., "Know your account: Double graph inference-based account de-anonymization on ethereum," in *Proc. IEEE 41st Int. Conf. Data Eng.*, 2025, pp. 1305–1318.
- [20] T. Kipf, "Semi-supervised classification with graph convolutional networks," 2016, *arXiv:1609.02907*.
- [21] J. Wu, K. Lin, D. Lin, B. Zhang, Z. Wu, and J. Su, "Safeguarding blockchain ecosystem: Understanding and detecting attack transactions on cross-chain bridges," in *Proc. ACM Web Conf.*, 2025, pp. 4902–4912.
- [22] J. Zhang, J. Gao, Y. Li, Z. Chen, Z. Guan, and Z. Chen, "Xscope: Hunting for cross-chain bridge attacks," in *Proc. 37th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2022, pp. 1–4.
- [23] S. Guan and K. Li, "Characterizing ethereum address poisoning attack," in *Proc. 2024 ACM SIGSAC Conf. Comput. Commun. Secur.*, 2024, pp. 986–1000.
- [24] B. Wang, X. Yuan, L. Duan, H. Ma, C. Su, and W. Wang, "DeFiScanner: Spotting DeFi attacks exploiting logic vulnerabilities on blockchain," *IEEE Trans. Computat. Social Syst.*, vol. 11, no. 2, pp. 1577–1588, Apr. 2024.
- [25] W. Chen, X. Guo, Z. Chen, Z. Zheng, and Y. Lu, "Phishing scam detection on ethereum: Towards financial security for blockchain ecosystem," in *Proc. Int. Joint Conf. Artif. Intell.*, 2020, pp. 4456–4462.
- [26] D. Zhang, J. Chen, and X. Lu, "Blockchain phishing scam detection via multi-channel graph classification," in *Proc. Int. Conf. Blockchain Trustworthy Syst.*, 2021, pp. 241–256.
- [27] L. Su et al., "Evil under the sun: Understanding and discovering attacks on ethereum decentralized applications," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 1307–1324.
- [28] Z. Wu, J. Wu, H. Zhang, Z. Zheng, and W. Wang, "Hunting in the dark forest: A pre-trained model for on-chain attack transaction detection in web3," in *Proc. ACM Web Conf.*, 2025, pp. 4519–4530.
- [29] S.-H. Wang, C.-C. Wu, Y.-C. Liang, L.-H. Hsieh, and H.-C. Hsiao, "Promutor: Detecting vulnerable price oracles in DeFi by mutated transactions," in *Proc. IEEE Eur. Symp. Secur. Privacy Workshops*, 2021, pp. 380–385.
- [30] S. Wu et al., "DeFiRanger: Detecting DeFi price manipulation attacks," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 4147–4161, Jul./Aug. 2023.
- [31] "Surge token reentrancy attack on xsurge," Aug. 2021. [Online]. Available: <https://bscscan.com/tx/0x7e2afec08464e8e0118368cb933dc64ed9ce36445ecf9c49cacb970ea78531d2>
- [32] Crypto Burgers Team, "Crypto burgers arbitrary burn vulnerability and flash loan attack," Jan. 2022. [Online]. Available: <https://medium.com/cryptoburgernft/it-has-been-6-days-since-the-attack-17fc253a57b5>
- [33] "Cream finance vault manipulation: Second flash loan attack," Oct. 2021. [Online]. Available: <https://etherscan.io/tx/0x0fe2542079644e107cbf13690eb9c2c65963ccb79089ff96bfaf8dced2331c92>
- [34] "Aperocket: Flash loan attack on cake vault," Jul. 2021. [Online]. Available: <https://bscscan.com/tx/0x701a308fba23f9b328d2cbb6c7b245f6c3063a510e0d5bc21d2477c9084f93e0>
- [35] "Platypus finance: Flash loan exploit and usp minting," Feb. 2023. [Online]. Available: <https://snowtrace.io/tx/0x1266a937c2ccd970e5d7929021eed3ec593a95c68a99b4920c2efa226679b430>
- [36] "Impossible finance: Flash loan attack via fake token LP manipulation," Jun. 2021. [Online]. Available: <https://explorer.bitquery.io/bsc/txs/calls?internal=false&contract=0xc7ca5478a41d58e73f0487b0f4084b6120aa11e6&method=641ccd83>
- [37] "Radiant capital: Flash loan exploit via precision and rounding vulnerability," Jan. 2024. [Online]. Available: <https://arbiscan.io/tx/0x1ce7e9a9e3b6dd3293c9067221ac3260858ce119ecb7c860eac28b2474c7c9b>
- [38] "bZx: Flash loan attack via wbt/eth price manipulation," Feb. 2020. [Online]. Available: <https://etherscan.io/tx/0xb5c8bd9430b6cc87a0e2fe110ece6bf527fa4f170a4bc8cd032f768fc5219838>
- [39] "Inverse finance: Second flash loan attack via oracle manipulation," Jun. 2022. [Online]. Available: <https://etherscan.io/tx/0x958236266991bc3fe3b77feaaeal20f172c0708ad01c7a715b255f218f9313c>
- [40] "Deus finance: Flash loan attack via oracle manipulation," Mar. 2022. [Online]. Available: <https://ftmscan.com/tx/0xe374495036fac18aa5b1a497a17e70f256c4d3d416dd1408c026f3f5c70a3a9c>
- [41] "Gamma strategies: Flash loan exploit via price manipulation and deposit proxy misconfiguration," Jan. 2024. [Online]. Available: <https://arbiscan.io/tx/0x025cf2858723369d606ee3abbc4ec01eab064a97cc9ec578bf91c6908679be75>
- [42] "Yearn.finance: Complex flash loan attack involving dydx, aave V2, compound, and curve," Feb. 2021. [Online]. Available: <https://etherscan.io/tx/0x59faab5a1911618064f1ffa1e4649d85c99cf9f0d64dcebbca1af7d7630da98b>
- [43] "Pancakebunny: Flash loan exploit on vaultfliptoflip vault," May 2021. [Online]. Available: <https://bscscan.com/tx/0x897c2de73dd55d7701e1b69ffb3a17b0f4801ced88b0c75fe1551c5fccc6a979>

- [44] "Beanstalk: Flash loan governance exploit resulting in 77m theft," Apr. 2022. [Online]. Available: <https://etherscan.io/tx/0xcd314668aaa9bbfbefaf1a0bd2b6553d01dd58899c508d4729fa7311dc5d33ad7>
- [45] "Kyberswap: Cross-chain flash loan exploit via price manipulation," Nov. 2023. [Online]. Available: <https://etherscan.io/tx/0x485e08dc2b6a4b3aeacdb89c3d18a37666dc7d9424961a2091d6b3696792f0f3>
- [46] QuickNode, "Quicknode: Blockchain infrastructure and API platform," 2025, Accessed: Oct. 25, 2025. [Online]. Available: <https://www.quicknode.com/>
- [47] DeFiLlama, "Ethereum chain overview on defillama," 2025, Accessed: Oct. 26, 2025. [Online]. Available: <https://defillama.com/chain/ethereum>
- [48] Rayo, "Rayo: Blockchain data infrastructure and analytics platform," 2025, Accessed: Oct. 26, 2025. [Online]. Available: <https://rayo.gg/>
- [49] Alchemy, "Alchemy: Web3 development and blockchain api platform," 2025, Accessed: Oct. 26, 2025. [Online]. Available: <https://www.alchemy.com/>
- [50] Etherscan, "Etherscan label cloud: Categorized ethereum address labels," 2025, Accessed: Oct. 26, 2025. [Online]. Available: <https://etherscan.io/labelcloud>
- [51] Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao, "Hypergraph neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 3558–3565.
- [52] BlockSec, "Blocksec metasuites: Comprehensive security and analysis platform for web3," 2025, Accessed: Oct. 26, 2025. [Online]. Available: <https://blocksec.com/metasuities>,
- [53] BlockSec Team, "Blocksec: Blockchain security infrastructure and research," 2025, Accessed: Oct. 27, 2025. [Online]. Available: <https://blocksec.com/>
- [54] "Fn transaction," 2025, Accessed: Oct. 27, 2025. [Online]. Available: <https://etherscan.io/tx/0x171072422efb5cd461546bfe986017d9b5aa427ff1c07ebe8acc064b13a7b7be>
- [55] BlockSec Team, "Blocksec phalcon explorer," 2025, Accessed: Oct. 27, 2025. [Online]. Available: <https://blocksec.com/explorer>,
- [56] "FP transaction," 2025, Accessed: Oct. 27, 2025. [Online]. Available: <https://bscscan.com/tx/0xdb624bf08fb70955f197c6f5b2ea950a7d9ca2048f02cb3f907da1931fecce0b>



Xinpeng Huang received the BS degree from the College of Computer Science, Nankai University, China, in 2022. Currently, he is currently working toward the PhD degree with the School of Artificial Intelligence, Beihang University through an integrated master-doctoral program. His research interests include blockchain security and artificial intelligence.



Wangjie Qiu received the BS and PhD degrees in mathematics from Beihang University, in 2008 and 2013. He is currently an associate professor with Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing, Beihang University. His research interests include information security, blockchain, and privacy computing. And he is also the deputy secretary general of the blockchain committee, China Institute of Communications. As the founding team, he successfully developed Chain-Maker, an international advanced blockchain technology system. He has been authorized a number of invention patents in the fields of information security, blockchain, and privacy computing. He won the first prize of China National Technology Invention in 2014.

ogy system. He has been authorized a number of invention patents in the fields of information security, blockchain, and privacy computing. He won the first prize of China National Technology Invention in 2014.



Wanqing Jie received the BS degree from the Department of Management Science and Engineering, Tianjin University of Finance and Economics, China, in 2020, and the MS degree from the Institute of Artificial Intelligence and Blockchain, Guangzhou University, China, in 2023. She is currently working toward the PhD degree with Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing, Beihang University, China. Her research interests include blockchain security and privacy computing.



Qing Xia received the PhD degree in computer science and technology from the Institute of Software, Chinese Academy of Sciences, Beijing, China, in 2022. She is currently an associate professor with the Institute of Software, Chinese Academy of Sciences. Her research interests include cryptocurrency, blockchain, and cyber security.



Qinnan Zhang received the PhD degree from the Central University of Finance and Economics, Beijing, China, in 2023. She is currently a postdoctoral fellow with Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing, Beihang University, Beijing. She was a visiting scholar with the Information Systems Technology and Design, Singapore University of Technology and Design, Singapore. She is currently an executive committee member of the CCF Blockchain Professional Committee. Her current research interests include blockchain security, federated learning, incentive mechanism, and edge intelligence.



Yuqiang Sun (Graduate Student Member, IEEE) received the PhD degree from the College of Computing and Data Science, Nanyang Technological University, Singapore in 2025. He is currently a research fellow with Nanyang Technological University.



Maoyi Xie received the bachelor's degree in cyber security from Tianjin University, in 2023. He is currently working toward the PhD degree with Nanyang Technological University, where his research focuses on software security and system security.



Yang Liu (Senior Member, IEEE) is a full professor with the College of Computing and Data Science, Nanyang Technological University, Singapore. He is also President's Chair professor in computer science and engineering, executive director for CyberSG Research & Development Programme Office, executive Director for Cyber Security Research Centre @ NTU.



Zhiming Zheng received the PhD degree in mathematics from the School of Mathematical Sciences, Peking University, Beijing, China, in 1987. He is currently a professor with the Institute of Artificial Intelligence, Beihang University, Beijing, China. His research interests include refined intelligence, blockchain, and privacy computing. He is one of the initiators of Blockchain-ChainMaker. He is a member of Chinese Academy of Sciences.